Scenario

Your team has been hired by a fictional analytics company — InsightFlow — to create a mini web-based data dashboard for tracking employee performance metrics.
The dashboard should dynamically render employee data, allow inline edits, and provide filters — all driven entirely by JavaScript DOM manipulation (no page reloads, no frameworks, no libraries).

Functional Requirements

Every group must implement all of the following using Vanilla JS + HTML + CSS only.

1. Dynamic Table Rendering: Fetch employee data from a static array of objects (provided in the script)
    - Example data: const employees = [{ id: 1, name: "Alice Johnson", role: "Developer", score: 88 }, { id: 2, name: "James Smith", role: "Designer", score: 73 }, { id: 3, name: "Fatou Kamara", role: "Project Manager", score: 91 }, { id: 4, name: "David Mwangi", role: "QA Engineer", score: 64 }];
    - Render this data in a styled table dynamically using DOM methods: createElement(), appendChild(), textContent, innerHTML (where appropriate).
    - No static HTML table should exist in the markup — it must be fully generated via JavaScript on page load.

2. Interactive Sorting
    - Each column header (Name, Role, Score) must be clickable.
    - Clicking toggles sorting order (ascending/descending).
    - Sorted results should immediately re-render in the DOM.

3. Search & Filter
    - A search bar filters employees by name as the user types.
    - A dropdown filter allows selection by role (e.g., show only Developers or Designers).
    - Filters should work in combination (i.e., typing “Alice” + filtering “Developer” should narrow results accordingly).

4. Inline Editing via Modal
    - Each row should have an “Edit” button.
    - Clicking “Edit” opens a modal (built dynamically via the DOM) allowing: Editing the employee’s role and score.
    - Saving updates should: Modify the underlying data array.
    - Immediately update the DOM to reflect the change.
    - Modal must include: Overlay background (clicking outside closes it).
    - Validation (e.g., score must be a number between 0 and 100).

5. Add & Delete Employees
    - An “Add Employee” button opens a form (could be a modal or inline section). The form captures name, role, and score.

- On submission:
  - A new employee is added to the array.
  - The table re-renders to include the new entry.
  - Each employee row should also include a “Delete” button that removes that entry from both the array and the DOM.

6. Visual Highlighting
   - Rows should visually indicate employee performance:
     - score >= 85 → green background
     - 70 ≤ score < 85 → yellow background
     - score < 70 → red background
   - These classes should be applied dynamically based on the data array — no manual CSS assignment per element.

7. Summary Section (Dynamic Stats)
   - At the bottom of the dashboard, display a summary card that automatically updates whenever data changes:
     - Total number of employees
     - Average score
     - Highest performing employee’s name
   - Each update (adding, editing, deleting) should trigger re-calculation and re-rendering of this section.

Technical Constraints

- Do not use any frameworks or libraries (no React, jQuery, etc.).
- All rendering must be handled via DOM APIs: createElement, appendChild, addEventListener, classList, removeChild, etc.
- Use modular functions for clarity (e.g., renderTable(), updateSummary(), openModal()).
- All states should reside in JavaScript memory (arrays/objects).
- The interface must update dynamically — no window.location.reload() allowed.